Week 8

Soft Constraints and Optimization

◎ Learning Objectives

By the end of this chapter, you will be able to:

- (**H1**) define an over-constrained problem and distinguish Max-CSP, weighted CSP (WCSP) and valued CSP, and explain when relaxing a hard-constraint CSP into one of these soft-constrained forms is the right response to infeasibility, rather than declaring the problem unsolvable;
- (**H2**) formulate and implement a constraint optimization problem's objective function in OR-Tools CP-SAT – including reified Boolean "violation" variables tied to a soft constraint through `OnlyEnforceIf` and a weighted linear objective built from them;
- (**H3**) carry out lexicographic multi-objective optimization through sequential CP-SAT solves (pinning each stage's optimum before the next), and determine Pareto dominance to build a Pareto front for a small set of candidate solutions under two competing objectives;
- (**H4**) build a complete soft-constraint model – combining hard feasibility/coverage constraints with weighted soft preferences via penalty variables and soft global constraints – for a realistic over-constrained scheduling scenario (timetabling or nurse-preference rostering);
- (**H5**) translate a real over-constrained scenario into a verified, running CP-SAT soft-constraint program and interpret its objective value and solution against the formal Max-CSP/WCSP/ valued-CSP definitions (H1) – closing the loop from theory to a working solver run, exactly as Week 1's H5 did for hard CSPs.

Weeks 1–6 asked whether a solution exists to a hard-constraint CSP, and Week 7 asked for the best assignment under a linear objective when a solution is not in question. This chapter asks a harder, more realistic question that sits between the two: what do you do when a problem's hard constraints cannot all be satisfied at once? This week's tool is OR-Tools CP-SAT only – no Gurobi, no MiniZinc – and it takes the objective-function machinery Week 7 introduced for `gurobipy`/MIP and shows the CP-SAT analogue, then goes one step further with reification, penalty variables and multi-objective trade-offs that a plain linear MIP objective does not, by itself, make visible.

This is also the last content week of Midterm 1's scope. Week 9 is a numbered-

topic-free Midterm 1 Review, covering Weeks 1–8 with no new topic; this week's over-constrained timetabling and nurse-preference models are deliberately built to be strong review material, since building them exercises the formal CSP vocabulary (Week 1), modeling choices (Week 1), global constraints (Week 2), CP-SAT search/time-limit parameters (Week 3), and objective functions (Week 7) all at once, inside one running program.

Four units carry this chapter. Unit 1 defines an over-constrained problem and its three graded relaxations – Max-CSP, weighted CSP and valued CSP – entirely on paper, sharpening Week 1's CSP-vs-COP distinction into its first concrete instance. Unit 2 turns that paper definition into running CP-SAT code: `model.Minimize()`/`model.Maximize()`, reification and `OnlyEnforceIf`, and a weighted-violation objective built from reified Booleans. Unit 3 shows why a single weighted-sum objective can hide a trade-off, and gives you two tools for handling more than one objective at once – lexicographic optimization and Pareto dominance/fronts. Unit 4 assembles every idea from Units 1–3 into one complete soft-constraint model on a second worked instance, nurse-preference scheduling, combining hard feasibility/coverage constraints with weighted soft preferences through penalty variables and soft global constraints.

This chapter deliberately stays out of several topics that later weeks own outright. It does not diagnose *why* a genuinely infeasible hard-constraint set is infeasible – no unsat cores, no infeasibility proofs – that is Week 15's entire subject; this week's response to "no schedule satisfies every hard rule" is to relax some rules to soft ones, not to explain the infeasibility itself. It uses only plain integer/ Boolean decision variables and reified linear constraints – no interval variables, no `NoOverlap`, no `Cumulative` scheduling API; that machinery for the same problem families arrives in Weeks 11–12. It introduces no CP–MIP hybrids or decomposition – that is Week 12. And it says nothing about LP/MIP duality, shadow prices, or ranging – Week 10 asks a different question, "how much would relaxing a hard constraint's right-hand side change the optimal objective," that only makes sense for LP/MIP models, not this week's CP-SAT soft-constraint models.

8.1 Over-Constrained Problems: Max-CSP, Weighted CSP, Valued CSP

1 • Pre-instruction — Why It Matters

Week 1 asked you to schedule three talks, T1–T3, into two time slots so that T1 and T2 – who share an author – never landed in the same slot: a solvable CSP with room to spare. Now suppose two more talks join the program, T4 and T5, and between them T1, T3 and T4 all share an author with someone else on the list. With only two slots, some shared-author pair is now unavoidably double-booked, no matter how you assign them. Do you tell the organizers "no valid schedule exists" and stop, or do you hand them the schedule that breaks the fewest rules?

1 • Pre-instruction — Prior Knowledge

Week 1 Unit 1 gave you the formal CSP triple – variables, domains, constraints – together with the CSP-vs-COP distinction: a plain CSP asks only whether a

solution exists, a constraint optimization problem (COP) adds an objective on top. This unit does not change what a constraint or a solution *is*; it changes what you do when no assignment satisfies every constraint at once. The five-talk instance above directly extends Week 1 Unit 1's own three-talk conference-scheduling example.

Every CSP you built through Week 5 came with an implicit promise: if a solution exists, the solver eventually finds it, and if none exists, the solver reports `INFEASIBLE` and stops. That promise is fine as far as it goes, but it says nothing useful about what to *do* once infeasibility is confirmed. A conference organizer does not want to hear "no schedule exists" and cancel the event; a lab manager sharing two microscopes among four incompatible research groups does not want to hear "no booking plan exists" and give up. This unit gives that situation a name and a family of formal responses.

8.1.1 Over-constrained problems: when `INFEASIBLE` is not the end

over-constrained problem: A constraint satisfaction problem whose hard constraints cannot all be satisfied by any complete assignment.

A CSP whose hard constraints cannot all be satisfied simultaneously by any complete assignment is called **over-constrained problem**. The five-talk instance above is exactly this: list its shared-author constraints as T1–T3, T1–T4, T3–T4 and T2–T5, and notice that T1, T3 and T4 form a triangle of pairwise "must differ" rules among themselves. With only two slots, some pair inside that triangle is forced to share a slot no matter how you assign the three talks – a short pigeonhole argument, not something that needs a solver to discover.

Definition: Over-constrained problem

A constraint satisfaction problem whose hard constraints cannot all be satisfied by any complete assignment.

Analogy

An over-booked event venue trying to seat every guest's request is the same situation: when two guests both insist on the same table and only one can have it, the venue does not cancel the event – it accepts the seating plan that honours the most requests, or costs the fewest refused requests, the same move this unit's tools make on constraints instead of seating requests.

Declaring a problem over-constrained and stopping there wastes the work already invested in modelling it. The rest of this unit relaxes the requirement that *every* hard constraint hold, and replaces it with a graded, comparative question: among all the assignments that exist, which one is *least bad*?

8.1.2 Max-CSP: count violations, minimize them

Max-CSP: The relaxation of an over-constrained CSP in which every constraint becomes soft and the goal is the assignment satisfying the largest number of constraints, equivalently violating the fewest.

The simplest relaxation treats every constraint as soft and asks for the assignment that satisfies the largest number of them – equivalently, the one that violates the fewest. This relaxation is called **Max-CSP**.

Definition: Max-CSP

The relaxation of an over-constrained CSP in which every constraint becomes soft and the goal is the assignment satisfying the largest number of constraints, equivalently violating the fewest.

A common misconception at this point is assuming that Max-CSP somehow weakens the model by "giving up" on constraints. It does not throw any constraint away – every constraint still counts against a candidate assignment exactly the way it did before – it only changes the pass/fail question ("are all constraints satisfied?") into a counting question ("how many are?"), and asks for the assignment that maximizes that count. A CSP that happens to be satisfiable is simply the special case where a Max-CSP solve finds an assignment violating zero constraints.

Case Study 8.1: The five-talk instance, worked by hand

Take the four shared-author constraints listed above: T1–T3, T1–T4, T3–T4, T2–T5. Try the assignment T1 = morning, T2 = morning, T3 = afternoon, T4 = afternoon, T5 = afternoon. Check each pair: T1 and T3 differ (morning vs. afternoon, satisfied); T1 and T4 differ (satisfied); T3 and T4 are *both* afternoon (violated); T2 and T5 differ (satisfied). This assignment violates exactly one constraint, T3–T4.

Now try a different assignment: T1 = morning, T3 = morning, T4 = morning, T2 = afternoon, T5 = afternoon. T1 and T3 are both morning (violated); T1 and T4 are both morning (violated); T3 and T4 are both morning (violated – though this is the same underlying triangle already double-counted by the first two); T2 and T5 differ (satisfied). Counting distinct violated *pairs* in this second assignment gives three violations (T1–T3, T1–T4, T3–T4 all fail at once, since all three talks share the morning slot), strictly worse than the first assignment's one violation. A Max-CSP objective prefers the first assignment, since one violated constraint beats three, even though neither assignment is a "clean" CSP solution in Week 1's sense.

Figure 8.1 draws the four shared-author constraints as a graph over the five talks, using the real weighted-optimal assignment CP-SAT reports for this instance (Unit 2 below runs the code that produces it): T1 and T3 both land in the morning slot, so the T1–T3 edge is the one drawn violated, while all three remaining edges are drawn satisfied. Exactly one of the four "must differ" constraints breaks in this optimal solution, matching the one-violation assignment worked by hand above.

8.1.3 Weighted CSP: not every violation costs the same

Max-CSP treats every broken constraint as equally bad – one violation is one violation, regardless of which constraint it is. Real problems rarely work that way: missing a strongly preferred rule can matter far more than missing a mildly preferred one. A **weighted CSP (WCSP)** attaches a numeric cost to each constraint's violation, and asks for the assignment minimizing the *total* cost of the constraints it breaks, rather than the raw count of them.

weighted CSP (WCSP): A generalization of Max-CSP in which each constraint carries a cost for being violated; the goal is the assignment minimizing total violation cost.

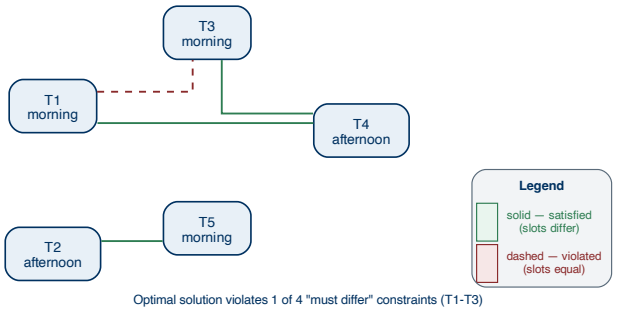


Figure 8.1: Constraint graph of the over-constrained five-talk timetable: satisfied vs. violated "must differ" edges in the real optimal solution.

Definition: Weighted CSP (WCSP)

A generalization of Max-CSP in which each constraint carries a cost for being violated; the goal is the assignment minimizing total violation cost.

Max-CSP is exactly the special case of a WCSP in which every constraint's weight equals 1 – minimizing a sum of ones counts violated constraints, which is precisely what Max-CSP already did. Nothing about the WCSP machinery changes when every weight happens to be equal; only the weights, once they start to differ, change which assignment the objective prefers.

Analogy

A household budget with priority-weighted bills makes the same trade-off: rent is unavoidable, so it counts as hard, never cut; a streaming subscription is discretionary, and cutting one subscription "costs" less regret than cutting another. The family minimizes total regret rather than treating every cut expense as equally bad – exactly how a WCSP weighs violations unequally instead of merely counting them.

Unit 4's nurse-preference model later this week is a direct WCSP instance: each nurse's stated shift preference carries its own weight, so one nurse's violated preference can cost more than another's, and the solver's objective adds up exactly those costs.

8.1.4 Valued CSP: the umbrella term

valued CSP: The general framework in which constraint satisfaction is a graded value rather than a yes/no, and the objective combines those values; WCSP is its most common instance in this course.

WCSP is the most common instance this course works with, but it is itself a special case of a broader framework: a **valued CSP** treats a constraint's "satisfaction" as a graded value rather than a strict yes/no, and combines those graded values – by summing costs, as a WCSP does, or by other combination rules the Bistarelli/Schiex family of formulations defines – into a single objective. This week works concretely with WCSP and Max-CSP throughout; valued CSP is given here only as the umbrella term under which they both sit, and is not developed further in this course.

Definition: Valued CSP

The general framework in which constraint satisfaction is a graded value rather than a yes/no, and the objective combines those values; WCSP is its most common instance in this course.

Put the four ideas of this unit together and Week 1's own CSP-vs-COP distinction sharpens into a concrete instance: a WCSP (or Max-CSP) is itself a constraint optimization problem, one whose objective is specifically "minimize weighted constraint violation." Nothing about the formal COP definition Week 1 gave you changes – this unit hands you the first genuinely worked example of it, on a problem that started life as a plain, unsolvable-as-stated CSP.

3 • Participation **Now You Try**

Take the five-talk, two-slot instance from the motivation hook above. List the shared-author constraints (T1–T3, T1–T4, T3–T4, T2–T5). Find one assignment of talks to {morning, afternoon} that violates exactly one of these constraints, and one that violates two. Which one would a Max-CSP objective prefer, and why?

Feedback. One assignment achieving exactly one violation: T1 = morning, T2 = morning, T3 = afternoon, T4 = afternoon, T5 = afternoon – only T3–T4 clash, since both land in the afternoon. A worse assignment such as T1 = morning, T3 = morning, T4 = morning, T2 = afternoon, T5 = afternoon violates two distinct constraints, T1–T3 and T1–T4, both clashing at once (T3–T4 also clashes here, but it is the same triangle already counted). Max-CSP counts violated constraints and prefers the assignment with fewer – one violation beats two, even though neither is a "clean" CSP solution in Week 1's sense.

Check Yourself 8.1

A small lab has 4 pieces of shared equipment booked by researchers R1–R4, and only 2 available time windows; more incompatible-equipment pairs exist than the two windows can keep apart, so no schedule satisfies every "do not overlap" rule at once. Restate this as a Max-CSP: what exactly does the objective now maximize (or minimize), and give one example schedule with 1 violation versus one with 2, explaining why Max-CSP prefers the first.

5 • Follow-through **Transfer — Where Else?**

The "relax hard constraints into weighted soft ones" move you just made is not specific to conference scheduling – Unit 4 later this week applies the identical idea to nurse-preference rostering, Week 12 applies it to full course/exam timetabling and nurse rostering with interval-variable machinery, and Week 15 revisits an over-constrained model exactly like this one to ask a different question: how do you tell a genuinely broken hard-constraint set, one no soft relaxation should paper over, apart from one that is merely expensive to satisfy fully?

8.2 Constraint Optimization Problems and Objective Functions in CP-SAT

1 • Pre-instruction **Why It Matters**

In Week 7 you wrote `model.setObjective(...)` to tell Gurobi "minimize total production cost." This unit asks the same question inside CP-SAT – "minimize total constraint-violation cost" – and the mechanics turn out almost identical, except CP-SAT forces you to be explicit about *which* Boolean variable stands for *which* violation before you can add it to an objective at all.

1 • Pre-instruction **Prior Knowledge**

Week 7's `model.setObjective()` / `Minimize`/`Maximize` pattern for `gurobipy` – an objective function is a linear expression over decision variables – and Week 3's CP-SAT time-limit/status vocabulary (`OPTIMAL` vs. `FEASIBLE` under a time limit) carry over directly into this unit, rather than introducing new solver-status concepts. Unit 1's Max-CSP/WCSP definitions supply the objective this unit builds: minimize total weighted violation.

Unit 1 defined *what* a WCSP objective minimizes – total weighted constraint-violation cost – entirely on paper, by counting violated pairs by hand. This unit turns that paper definition into a running CP-SAT program: how do you tell `model.Minimize()` to count violations at all, when the objective's argument has to be a linear expression over variables, not a true/false test on two talks' slots?

8.2.1 CP-SAT's optimization API: the direct analogue of Week 7

CP-SAT exposes exactly two optimization calls, `model.Minimize(expr)` and `model.Maximize(expr)`, where `expr` must be a linear integer expression over the model's variables – the direct CP-SAT analogue of Week 7's `gurobipy` `model.setObjective(...)`. A pure CSP (Weeks 1–6) never called either one, because it carried no objective at all; the moment a model calls `model.Minimize()` or `model.Maximize()`, it stops being a plain CSP and becomes a constraint optimization problem, in CP-SAT's own API, on top of the exact same variables and constraints a plain CSP would have used.

Reading a CP-SAT optimization run's outcome reuses Week 3's status vocabulary without change: `cp_model.OPTIMAL` means the objective's optimum is proven, while `cp_model.FEASIBLE` – returned when a time limit cuts search short – means a solution was found but optimality is not proven. The same `OPTIMAL`/`FEASIBLE` distinction Week 7 introduced for Gurobi's branch-and-bound MIP search now applies to CP-SAT's own branch-and-bound-with-propagation search, unchanged.

8.2.2 Reification: giving a condition a variable of its own

Here is the obstacle Unit 1's hand-counting glossed over: `model.Minimize()` needs a linear expression over the model's *variables*. The raw condition "`slot[T1] != slot[T3]`" is not itself a variable – it is a comparison, evaluated once when the constraint is built, not something `model.Minimize()` can add up across four different pairs. What the objective needs instead is a genuine 0/1 variable that tracks whether

each pair actually clashes in a given solution – and CP-SAT's mechanism for building exactly that variable is called **reification**.

reification: Turning a constraint's truth value into a Boolean decision variable that other parts of the model, in particular the objective, can reference.

Definition: Reification

Turning a constraint's truth value into a Boolean decision variable that other parts of the model, in particular the objective, can reference.

Analogy

Think of a household checklist scorecard: for each house rule, a checkbox flips to "broken" only if that specific rule was actually broken this week, and the household's weekly score is the sum of flipped checkboxes. A reified violation variable is exactly that one checkbox, tied to one specific rule, ready to be added into a total the way the scorecard adds flipped checkboxes into a weekly score.

CP-SAT builds this link through a call of the shape

```
model.Add(constraint).OnlyEnforceIf(b)
```

where the converse direction also matters, the negated literal `b.Not()` passed to the same call covers it. The mechanism linking a constraint to a Boolean literal, so that the constraint only holds when the literal is true (and, together with its negation, only fails to hold when the literal is false), is called **OnlyEnforceIf**. A reified violation variable always needs *both* directions, or the link is incomplete: one call forces the Boolean to 1 whenever the underlying condition holds, the other forces it to 0 whenever the condition does not hold.

OnlyEnforceIf: The CP-SAT mechanism linking a constraint to a Boolean literal so the constraint only holds when the literal is true, and, with its negation, only fails to hold when the literal is false.

Case Study 8.2: Reifying T1–T3's violation Boolean

Take the five-talk instance's `slot` variables, one per talk, domain {0, 1} for morning/afternoon:

```
slots = {talk: model.NewIntVar(0, 1, f"slot_{talk}") for talk in TALKS}
return slots
```

The constraint "T1 and T3 differ" needs a reified Boolean `viol_T1_T3` that equals 1 exactly when T1 and T3 land in the *same* slot. Both directions of `OnlyEnforceIf`, applied to every shared-author pair at once:

```
violations = {}
for talk_a, talk_b in PAIRS:
    viol = model.NewBoolVar(f"viol_{talk_a}_{talk_b}")
    model.Add(slots[talk_a] == slots[talk_b]).OnlyEnforceIf(viol)
    model.Add(slots[talk_a] != slots[talk_b]).OnlyEnforceIf(viol.Not())
    violations[(talk_a, talk_b)] = viol
return violations
```

The first `model.Add(...)` forces `viol` to 1 whenever the two talks' slots are equal (a clash); the second forces it to 0 whenever the slots differ (no clash). Neither direction alone is enough, since a solver left free to choose `viol`'s value in the un-covered direction could set it to whatever value makes the objective look best, rather than what actually happened in the assignment.

Common Pitfall

A frequent mistake when reifying a violation is writing only one of the two `OnlyEnforceIf` calls – typically the "forces it to 1 when violated" direction, since that direction feels like the whole point. Leaving out `OnlyEnforceIf(viol.Not())` lets the solver set `viol = 0` even when the talks genuinely clash, because nothing in the model forbids it: an unconstrained Boolean is free to take either value whenever no constraint pins it down, and `model.Minimize()` will happily exploit that freedom to make the objective look smaller than it really is, silently undercounting violations. Both directions are required every time a violation Boolean is built.

8.2.3 Building the weighted-violation objective

With one reified Boolean per soft constraint in hand, Unit 1's WCSP objective becomes a single line: one weight per Boolean, summed and handed to `model.Minimize()`.

```
weight = 1
total_violations = sum(weight * viol for viol in violations.values())
model.Minimize(total_violations)
return total_violations
```

For the Max-CSP case, every weight equals 1 – exactly Unit 1's observation that Max-CSP is the WCSP special case where no violation costs more than any other. Solving and reading back the result:

```
solver = cp_model.CpSolver()
solver.parameters.num_workers = 1
solver.parameters.random_seed = 42
status = solver.Solve(model)
assignment = {talk: solver.Value(slots[talk]) for talk in TALKS}
total = solver.Value(total_violations)
print(f"Status: {solver.StatusName(status)}")
for talk in TALKS:
   print(f"{talk}: {SLOT_NAMES[assignment[talk]]}")
print(f"Total violations: {total}")
```

This is the real, captured output of the code above:

Expected Output

```
Status: OPTIMAL
T1: morning
T2: afternoon
T3: morning
T4: afternoon
T5: morning
Total violations: 1
```

`OPTIMAL` status means CP-SAT has *proven* that no assignment achieves fewer than 1 violation, not merely that it found an assignment with 1 – exactly the guarantee Week 7's branch-and-bound gave you for a MIP's optimum. Reading the assignment back against Figure 8.1 confirms Unit 1's hand-worked count: T1 and T3 are the only pair sharing a slot (both morning), so the T1–T3 edge is exactly the one the figure draws violated, and every other edge is satisfied.

3 • Participation **Now You Try**

For the five-talk instance, the constraint "T1 and T3 differ" needs a reified Boolean `viol_T1_T3` that is 1 exactly when T1 and T3 are scheduled in the *same* slot. Write the two `OnlyEnforceIf` calls (one for each direction) that correctly link `viol_T1_T3` to the equality `slot[T1] == slot[T3]`.

Feedback. `model.Add(slot[T1] == slot[T3]).OnlyEnforceIf(viol_T1_T3)` forces the violation Boolean to 1 whenever the two talks *do* clash, and `model.Add(slot[T1] != slot[T3]).OnlyEnforceIf(viol_T1_T3.Not())` forces it to 0 whenever they do not – both directions are needed, or the solver could set `viol_T1_T3 = 0` even when the talks actually clash, silently undercounting violations in the objective.

Check Yourself 8.2

In CP-SAT, why can't you write `model.Minimize(x != y)` directly to penalize `x` and `y` being equal? What API pattern from this unit is needed instead, and what does it do?

5 • Follow-through **Transfer — Where Else?**

Every soft-constraint model for the rest of this course – Unit 4's nurse preferences later this week, Week 12's timetabling/rostering, Week 14's routing/bin-packing cost trade-offs – is built from exactly this pattern: reify a condition, weight it, sum it into `model.Minimize()`. Once you can do it for one shared-author clash, you can do it for any soft rule you can state as a constraint.

8.3 Lexicographic and Multi-Objective Optimization, Pareto Fronts

1 • Pre-instruction **Why It Matters**

Minimizing total violations alone, on the five-talk instance, might return a schedule that is numerically optimal but crams four of the five talks into the afternoon, leaving the morning slot nearly empty – technically correct, but not what any organizer actually wants. A second, competing objective is needed, and the two objectives will not always agree on what "best" means.

1 • Pre-instruction **Prior Knowledge**

Unit 2's `model.Minimize()` pattern is the exact building block this unit reuses. Lexicographic optimization is that same pattern run twice, with the first run's optimal value fed forward as a new constraint before the second run starts – nothing new is needed on the CP-SAT API side, only a change in how you sequence two already-familiar solves.

Week 1 Unit 1's own practice-task feedback already hinted at a second objective for a talk-scheduling instance: "it would only become a COP if an objective were

added, e.g. 'minimize the number of afternoon slots used."' This unit finally cashes in that foreshadowing, alongside Unit 2's total-violation objective, on the exact same five-talk instance.

8.3.1 Why a single weighted sum can hide a trade-off

A weighted-sum objective, the kind Units 1–2 built, reduces every soft-constraint violation to one number. That compression is convenient, but it throws away information: two very different schedules can reach the identical optimal total-violation value while looking completely different in every other respect – one spreading its clashes evenly, another crowding every talk into a single slot. The single number cannot show you that difference happened; you need a second objective, examined alongside the first, to see it.

8.3.2 Lexicographic optimization: rank, then pin

Lexicographic optimization: Optimizing a ranked sequence of objectives in strict priority order, pinning each objective's optimal value as a hard constraint before optimizing the next.

Lexicographic optimization ranks a sequence of objectives by strict priority. Solve for objective 1 first, exactly Unit 2's `model.Minimize()` call, and record its optimal value, v_1^*. Then add `model.Add(objective1_expr == v1_star)` as a new hard constraint on a fresh model, and minimize objective 2 subject to that constraint. Repeat for further objectives if there are any. The first objective's optimum is never traded away for the second's – it is locked in, by construction, before the second solve even starts.

Definition: Lexicographic optimization

Optimizing a ranked sequence of objectives in strict priority order, pinning each objective's optimal value as a hard constraint before optimizing the next.

Analogy

Ranking apartment-hunting priorities in strict order works the same way: first, among all apartments within budget, find the cheapest; only among apartments tied on that cheapest price, pick the one with the shortest commute. The second criterion never overrides the first – a slightly more expensive apartment with a shorter commute never beats the cheapest one, no matter how much shorter its commute is.

Case Study 8.3: Two-stage lexicographic solve on the five-talk instance

Stage 1 reuses Unit 2's total-violations objective exactly:

```
model.Minimize(total_violations)
solver = cp_model.CpSolver()
solver.parameters.num_workers = 1
solver.parameters.random_seed = 42
solver.Solve(model)
v1_star = solver.Value(total_violations)
print(f"Stage 1 optimal total violations: {v1_star}")
```

Stage 2 rebuilds the model – the same variables and violation Booleans, freshly created – pins stage 1's optimal value as a hard constraint, and only then declares a second objective, the number of talks scheduled in the afternoon slot:

```
model.Add(total_violations == v1_star)
afternoon_count = sum(slots[talk] for talk in TALKS)

model.Minimize(afternoon_count)
solver = cp_model.CpSolver()
solver.parameters.num_workers = 1
solver.parameters.random_seed = 42
solver.Solve(model)
afternoon_star = solver.Value(afternoon_count)
print(f"Stage 2 optimal afternoon count (violations pinned at {v1_star})
    : {afternoon_star}")
for talk in TALKS:
   print(f"{talk}: {SLOT_NAMES[solver.Value(slots[talk])]}")
```

The real, captured output of both stages together:

Expected Output

```
Stage 1 optimal total violations: 1
Stage 2 optimal afternoon count (violations pinned at 1): 2
T1: morning
T2: morning
T3: morning
T4: afternoon
T5: afternoon
```

Stage 1 proves the same one-violation optimum Unit 2's run already found. Stage 2, forced to keep that value fixed at exactly 1, finds the smallest possible afternoon count consistent with it: 2, achieved by putting T1, T2 and T3 in the morning and T4, T5 in the afternoon. Check the violation count directly against this new assignment: T1–T3 are both morning (violated, the one constraint stage 1 already accepted as unavoidable), T1–T4 and T3–T4 differ (satisfied), T2–T5 differ (satisfied) – exactly 1 violation, never traded away for the smaller afternoon count.

Tip

Notice why stage 2 needs a *fresh* model rather than reusing stage 1's model object directly: once a CP-SAT model has been solved, adding a new objective to the same model object does not cleanly "continue" the search the way you might expect – rebuilding the variables and constraints from scratch for stage 2, then adding the pinning constraint before the second `model.Minimize()` call, keeps the two stages unambiguous and easy to verify independently.

8.3.3 Pareto dominance and the Pareto front

Lexicographic optimization always hands you a single winner. A different tool, useful when you do not want to commit to a strict priority order in advance, instead shows you every reasonable trade-off point at once. For two objectives, a candidate solution A is said to exhibit **Pareto dominance** over candidate B if A is at least as good as B on *both* objectives and strictly better on at least one.

Pareto dominance: A candidate solution A dominates candidate B if A is at least as good as B on every objective and strictly better on at least one.

Definition: Pareto dominance

A candidate solution A dominates candidate B if A is at least as good as B on every objective and strictly better on at least one.

Pareto front: The set of candidate solutions that no other candidate dominates, under a given set of objectives.

The set of candidates no other candidate dominates is the **Pareto front** – unlike lexicographic optimization, a Pareto front does not pick a single winner; it hands the decision-maker every reasonable trade-off point, and lets a human (an organizer, a manager) choose among them using criteria the model itself never had to state.

Definition: Pareto front

The set of candidate solutions that no other candidate dominates, under a given set of objectives.

Analogy

Compare apartment listings on price and commute time: a listing is only worth shortlisting if no other listing beats it on both price and commute time at once. The shortlist of such not-beaten-on-both listings is the Pareto front – a cheaper-but-farther listing and a pricier-but-closer listing can both stay on the list at the same time, since neither dominates the other.

Case Study 8.4: Enumerating the five-talk instance's Pareto front

The five-talk instance is small enough to enumerate completely – 5 talks, 2 slots each, 32 possible assignments – using Week 1 Unit 4's enumerate-all-solutions technique, reused as a solution callback that records each candidate's (violations, afternoon count) pair:

```
def __init__(self, slots: dict[str, cp_model.IntVar]) -> None:
    super().__init__()
    self._slots = slots
    self.candidates: list[tuple[int, int]] = []

def OnSolutionCallback(self) -> None:
    assignment = {talk: self.Value(self._slots[talk]) for talk in TALKS}
    violations = sum(1 for a, b in PAIRS if assignment[a] == assignment[b
        ])
    afternoon = sum(assignment.values())
    self.candidates.append((violations, afternoon))
```

The dominance test itself is a small, pure Python function, matching the definition above almost word for word:

```
at_least_as_good = candidate_a[0] <= candidate_b[0] and candidate_a[1]
    <= candidate_b[1]
strictly_better = candidate_a[0] < candidate_b[0] or candidate_a[1] <
    candidate_b[1]
return at_least_as_good and strictly_better
```

Filtering every enumerated candidate against every other candidate keeps only the non-dominated ones:

```
unique_candidates = sorted(set(candidates))
front = [
```

```
    candidate
    for candidate in unique_candidates
    if not any(
        dominates(other, candidate) for other in unique_candidates if other
            != candidate
    )
]
return front
```

The real, captured output, listing all 12 distinct (violations, afternoon count) pairs the enumeration reaches, with the Pareto front marked:

Expected Output

```
All candidates (violations, afternoon_count):
  (1, 2) <- Pareto front
  (1, 3)
  (2, 1) <- Pareto front
  (2, 2)
  (2, 3)
  (2, 4)
  (3, 1)
  (3, 4)
  (4, 0) <- Pareto front
  (4, 2)
  (4, 3)
  (4, 5)
Pareto front:
  (1, 2)
  (2, 1)
  (4, 0)
```

Three candidates survive the dominance filter: $(1, 2)$ – exactly the lexicographic solution worked out above – $(2, 1)$, and $(4, 0)$. Check one dominated example by hand: $(1, 3)$ is dominated by $(1, 2)$, since $(1, 2)$ matches it on violations (both 1) and strictly beats it on afternoon count ($2 < 3$); $(1, 3)$ therefore cannot belong to the front. By contrast, $(2, 1)$ is *not* dominated by $(1, 2)$: $(2, 1)$ is worse on violations ($2 > 1$) but strictly better on afternoon count ($1 < 2$), so neither candidate is at least as good as the other on both objectives at once – the definition of dominance simply does not apply between them, and both stay on the front.

Figure 8.2 plots all 12 candidates on the two objectives, with the three Pareto-optimal points highlighted and connected by a step line tracing the front, and every dominated point drawn smaller and unconnected.

3 • Participation **Now You Try**

Suppose minimizing total violations on the five-talk instance gives an optimal value of 1. Write, in order, the two steps of a lexicographic optimization that first fixes this value and then minimizes the number of talks scheduled in the afternoon slot.

Feedback. Step 1 – solve `model.Minimize(total_violations)`, obtaining $v_1^* = 1$. Step 2 – build a fresh model (or add to the same one)

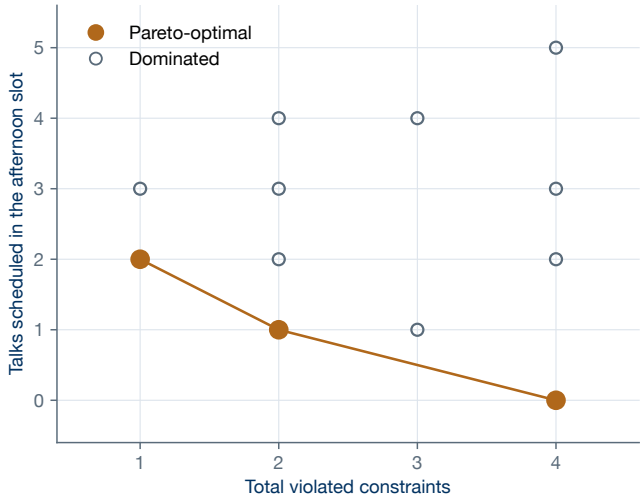


Figure 8.2: Pareto dominance and the Pareto front for the five-talk over-constrained timetable.

with the hard constraint `model.Add(total_violations == 1)`, then solve `model.Minimize(afternoon_count)` on that constrained model; the reported afternoon count is now the smallest possible *without* sacrificing the already-proven-optimal violation count of 1.

Check Yourself 8.3

You minimize total violations first and obtain an optimal value of 2. Why is it wrong to then independently minimize a second objective, on the very same model, without first adding any new constraint, if your goal is a true lexicographic optimum?

5 • Follow-through **Transfer — Where Else?**

Week 12's nurse-rostering model will need more than one fairness-related objective – total unfilled shifts, and how evenly weekends are distributed among nurses – and the lexicographic recipe and the Pareto-dominance test you just practiced are exactly the tools that model reuses, not a new technique to relearn. Week 14's scenario/what-if comparisons across capacity and cost also lean on the same "compare candidates on more than one number" instinct this unit builds.

8.4 Penalty Variables and Soft Global Constraints

1 • Pre-instruction **Why It Matters**

A hospital ward's real nurse roster has rules that must never break – every shift needs coverage, no nurse works two shifts back to back without rest – and preferences that matter but can be sacrificed when necessary – a nurse who would rather not work evenings. Bundling both kinds of rule into a single solver run,

so the hard rules are always respected and the preferences are honoured as much as possible, is where every idea from this week finally comes together into one program you can actually run.

1 • Pre-instruction **Prior Knowledge**

Unit 1's weighted-violation framing (WCSP), Unit 2's reification/ `OnlyEnforceIf`/`model.Minimize()` API, and Week 2's global-constraint vocabulary – what `AllDifferent`, `NoOverlap` and `Cumulative` each propagate – are the three ingredients this unit combines. Nothing here introduces a new solver mechanism; it combines three already-learned ones into a single soft global-constraint model.

Units 1–3 built the full soft-constraint toolkit one piece at a time, on a single running example: define an over-constrained problem's relaxed form (Unit 1), reify a violation and weight it into an objective (Unit 2), rank or compare more than one objective (Unit 3). This unit assembles all three pieces into one complete model on a second, independently worked instance – nurse-preference scheduling – and, in doing so, shows you a slightly more general recipe than a single pairwise "must differ" constraint: a whole *global* constraint can be made soft, not just a pairwise one.

8.4.1 Soft global constraints: the general recipe

Take any of Week 2's global constraints – `AllDifferent`, `NoOverlap`, `Cumulative` – and, instead of adding it directly as a hard rule, reify "does this instance of the constraint hold" into a Boolean and add a weighted penalty term to the objective when it does not. This reified Boolean, tracking whether (and by how much) one specific soft constraint is broken in a given assignment, is called a **penalty variable**.

penalty variable: A reified Boolean (or small integer) variable representing whether, and by how much, a specific soft constraint is violated in a given assignment.

Definition: Penalty variable

A reified Boolean (or small integer) variable representing whether, and by how much, a specific soft constraint is violated in a given assignment.

Unit 2's `viol_T1_T3` was already a penalty variable, tied to a single pairwise inequality. The same reification pattern – link a Boolean to a condition with `OnlyEnforceIf` in both directions, then weight and sum it into the objective – applies unchanged when the condition being reified is a whole global constraint's satisfaction rather than one pairwise comparison. A **soft global constraint** is exactly that: a global constraint whose satisfaction is reified into a penalty term instead of being enforced as a hard rule.

soft global constraint: A global constraint, such as `AllDifferent`, `NoOverlap` or `Cumulative`, whose satisfaction is reified into a penalty term rather than enforced as a hard rule.

Definition: Soft global constraint

A global constraint, such as `AllDifferent`, `NoOverlap` or `Cumulative`, whose satisfaction is reified into a penalty term rather than enforced as a hard rule.

Analogy

A shared lab's equipment sign-up sheet with a posted norm instead of a locked calendar makes the same trade-off: "please avoid double-booking the microscope" is requested, not physically enforced. Double-bookings that do happen get tallied and discouraged – a penalty – rather than made impossible outright the way a locked calendar, a hard constraint, would make them.

8.4.2 Worked case: nurse-preference scheduling

A 3-nurse, 2-day, 2-shift-per-day roster illustrates the full recipe. Two rules stay hard, no matter what: every shift-slot needs exactly one nurse covering it, and no nurse works both shifts on the same day.

```
for day in DAYS:
   for shift_type in SHIFT_TYPES:
      model.Add(sum(shift[nurse, day, shift_type] for nurse in NURSES) == 1)
for nurse in NURSES:
   for day in DAYS:
      model.Add(sum(shift[nurse, day, shift_type] for shift_type in
          SHIFT_TYPES) <= 1)
```

Neither loop reifies anything – these constraints are added with a plain `model.Add(...)` and no `OnlyEnforceIf`, exactly because a hard rule must always hold; there is nothing to weigh a violation of, since none is allowed to exist in the first place.

The soft part of the model is each nurse's stated shift preference. In this instance, all three nurses would rather not work evenings, but the strength of that preference differs by nurse: N1 objects most strongly (weight 3), N2 moderately (weight 2), N3 barely at all (weight 1). This differs slightly from an earlier sketch of the scenario, which imagined one nurse preferring to avoid evenings and a different nurse preferring mornings; the actual running model below gives every nurse the *same* kind of preference – avoid evenings – and lets the weights alone determine who ends up absorbing the unavoidable violation, which is in fact a cleaner illustration of a WCSP's weighting mechanism than two differently *shaped* preferences would have been. Since both evening slots must be covered by someone every day, at least one nurse's preference is unavoidably violated – the same "some clash is unavoidable" structure Unit 1 built the entire WCSP framing around, now appearing inside a soft global-constraint model instead of a pairwise timetabling one.

```
pref_violated = {}
for nurse in NURSES:
   for day in DAYS:
      violated = model.NewBoolVar(f"pref_violated_{nurse}_{day}")
      model.Add(shift[nurse, day, "evening"] == 1).OnlyEnforceIf(violated)
      model.Add(shift[nurse, day, "evening"] == 0).OnlyEnforceIf(violated.Not
          ())
      pref_violated[(nurse, day)] = violated
return pref_violated
```

Both `OnlyEnforceIf` directions appear again, exactly as Unit 2 required: the first `model.Add(...)` pins `violated` to 1 whenever the nurse actually works that day's evening shift, the second pins it to 0 whenever they do not.

Common Pitfall

It is tempting to think "all three nurses share the same preference, so one shared penalty variable should cover all of them." It should not: a penalty variable always ties to one specific nurse's stated preference on one specific day, never a pooled or averaged one. Collapsing three distinct preferences into a single shared Boolean would make it impossible for the objective to route the unavoidable violation toward the *cheapest* nurse to disappoint – which is precisely the outcome the weighted objective below is built to find.

The weighted objective sums each nurse's weight against their own violated-preference Boolean:

```
total_penalty = sum(
    PREFERENCE_WEIGHTS[nurse] * violated for (nurse, _day), violated in
        pref_violated.items()
)
model.Minimize(total_penalty)
return total_penalty
```

Solving and reporting the schedule alongside the total penalty:

```
solver = cp_model.CpSolver()
solver.parameters.num_workers = 1
solver.parameters.random_seed = 42
status = solver.Solve(model)
print(f"Status: {solver.StatusName(status)}")
for nurse in NURSES:
    row = []
    for day in DAYS:
        worked = [s for s in SHIFT_TYPES if solver.Value(shift[nurse, day, s])]
        row.append(f"{day}:{worked[0] if worked else 'off'}")
    print(f"{nurse}: {' '.join(row)}")
print(f"Total penalty: {solver.Value(total_penalty)}")
```

The real, captured output of the completed model:

Expected Output

```
Status: OPTIMAL
N1: Mon:off  Tue:morning
N2: Mon:morning  Tue:off
N3: Mon:evening  Tue:evening
Total penalty: 2
```

Read this against the weights: N1 and N2 never work an evening shift at all in this optimal schedule, so both of their preferences are fully honoured; N3, the cheapest nurse to disappoint (weight 1), absorbs *both* evening slots, for a penalty of $1 \times 2 = 2$. No cheaper assignment of the two unavoidable evening slots exists – routing even one of them to N1 or N2 instead would cost at least 2 or 3 per slot, strictly worse than leaving both with N3. This is exactly the weighted objective doing what Unit 1 promised: not eliminating the unavoidable violation, but routing it to where it costs the least.

Figure 8.3 shows this same schedule as a nurse-by-shift grid, colouring each filled cell by whether it honoured or violated that nurse's stated preference.

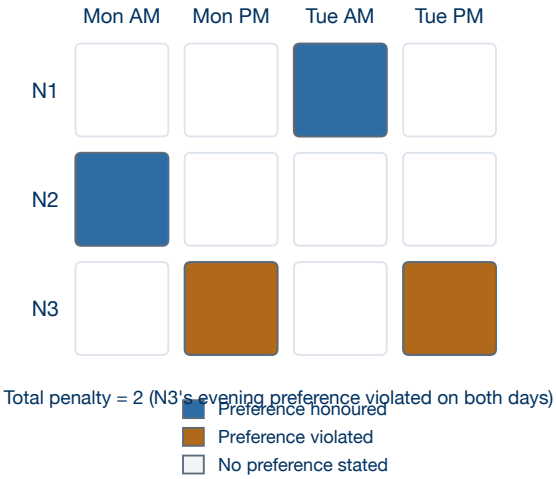


Figure 8.3: Optimal schedule for the 3-nurse soft-constraint instance, coloured by preference honoured/violated.

Tip

A hard-constraint-only version of this same roster – coverage and one-shift-per-day rules alone, no preferences at all – can still turn out infeasible if those rules cannot be met by any assignment. Diagnosing *that* kind of failure, rather than relaxing it into a soft objective, is Week 15's subject; this unit only asks you to notice the distinction exists between a genuinely broken hard-constraint set and one that is merely expensive to satisfy in full, the same distinction Unit 1's transfer note already raised for timetabling.

3 • Participation **Now You Try**

A partially written CP-SAT nurse-preference script has its hard coverage constraint in place (exactly one nurse per shift-slot) but is missing the reified penalty variable and `OnlyEnforceIf` calls for one nurse's stated evening-shift preference. Add the missing reification and the missing term in the objective's weighted sum, then run the script and report the total penalty.

Feedback. Add `model.Add(shift[nurse][slot] == 1).OnlyEnforceIf(pref_violated)` and `model.Add(shift[nurse][slot] == 0).OnlyEnforceIf(pref_violated.Not())` for the evening slot in question, then include `weight * pref_violated` as one more term in the existing `model.Minimize(sum(...))` call. Running the completed model on this week's 3-nurse instance reports the real total penalty: **2** – both evening slots routed to N3, the cheapest nurse to disappoint, read directly from `nurse-preferences-solution.txt` rather than guessed in advance.

Check Yourself 8.4

Two nurse schedules both satisfy every hard coverage rule. Schedule A has a strictly lower total preference-violation penalty than Schedule B. Does that guarantee Schedule A is more "fair" to the nurses? Give one reason a single

weighted-penalty total might not capture fairness.

5 • Follow-through **Transfer — Where Else?**

Weeks 11–12 build interval-variable, hard-constraint scheduling models (`NoOverlap`, `Cumulative`) for job-shop, flow-shop and RCPSP problems that structurally resemble this week's nurse-preference model; Week 12's nurse rostering with soft constraints is a direct extension of the model you just completed, now with interval variables standing in for shifts. Week 15 returns to this exact soft-constraint model to ask what changes if the hard part alone – coverage and rest rules, no preferences at all – turns out to have no feasible schedule.

Chapter Summary

Summary

This chapter gave you the vocabulary and the CP-SAT machinery for problems whose hard constraints cannot all be satisfied at once. Unit 1 defined an over-constrained problem and its graded relaxations: Max-CSP, which counts violated constraints and minimizes them; weighted CSP, which attaches a cost to each constraint's violation and minimizes total cost; and valued CSP, the umbrella framework both sit under. Unit 2 turned that paper definition into running code, showing how `model.Minimize()` needs a genuine variable rather than a raw condition, and how reification – `OnlyEnforceIf` in both directions – builds exactly that variable for a soft constraint. Unit 3 showed why a single weighted-sum objective can hide a trade-off, and gave you two tools for a second objective: lexicographic optimization, which ranks objectives by strict priority and pins each one's optimum before optimizing the next, and Pareto dominance, which instead hands you every non-dominated trade-off point as a Pareto front. Unit 4 assembled all three units into one complete model – hard coverage rules that never bend, and weighted soft preferences reified into penalty variables – on a nurse-preference rostering instance, and introduced the general recipe for turning any of Week 2's global constraints (`AllDifferent`, `NoOverlap`, `Cumulative`) into a soft global constraint. Week 9's Midterm 1 Review draws directly on this chapter's two running instances – the five-talk timetable and the 3-nurse roster – alongside five prior weeks' ideas, before Week 10 turns to a different kind of flexibility question entirely: LP/MIP duality and sensitivity analysis.

Check Yourself Answers

Question (Unit 1 – shared-equipment lab, Max-CSP restatement). Restated as a Max-CSP, the objective maximizes the number of satisfied "do not overlap" constraints among R1–R4's equipment bookings – equivalently, it minimizes the number violated. A schedule with 1 violation places every incompatible pair except one into different time windows, leaving only a single clashing pair; a schedule with 2 violations leaves two incompatible pairs clashing. Max-CSP prefers the 1-violation schedule because it has strictly fewer broken rules, with everything else about the

two schedules otherwise comparable – exactly the same counting argument the five-talk instance in this unit already walked through.

Question (Unit 2 – why a raw inequality cannot be minimized directly). `model.Minimize()` requires a linear integer expression over the model's variables, not a boolean truth-test; `x != y` is not itself a variable, it is a comparison evaluated once when written. The needed pattern is reification: create a Boolean variable and link it to the condition with `OnlyEnforceIf` in both directions – one call for when the condition holds, one for its negation – then use that Boolean, not the raw condition, inside the objective.

Question (Unit 3 – why the second objective needs the first pinned first). Without pinning `total_violations == 2` as a hard constraint before the second solve, the second minimization is free to trade away the first objective's optimality – for example, accepting 3 violations instead of 2 – in exchange for a better second-objective value, which is no longer a lexicographically optimal solution. The first optimum must be locked in as a hard constraint before the second objective is ever minimized; only then does the second solve search exclusively among assignments that already match the first objective's proven-optimal value.

Question (Unit 4 – lower total penalty vs. fairness). No – a single weighted sum can hide *how* the violations are distributed. Schedule A's lower total could still come from dumping every violation onto one nurse, while Schedule B spreads a slightly higher total more evenly across several nurses. Capturing that distinction needs a second, fairness-oriented objective – such as minimizing the maximum penalty any single nurse carries – and the lexicographic/Pareto tools from Unit 3, not a single weighted sum alone.